

Procesos de Wake

Cómo funciona Wake como empresa, extremo a extremo. Cuatro bloques: **Negocio**, **Contenido**, **Producto/Ingeniería** y **Finanzas**. Cada proceso indica disparador → pasos → responsable → sistemas.

Última actualización: 2026-07-11 · Responsable por defecto de todo: Emilio (ver `docs/ORGANIGRAMA.md`).

1. Negocio — del desconocido al cliente

1.1 Adquisición (funnel)

1. Tráfico entra por Landing (/), buy page pública de un programa (`CreatorProgramDetailScreen`) o campañas IG/ManyChat.
 2. Buy page vende **sin cuenta previa**: el comprador escribe solo su correo.
 3. Instrumentación en PostHog (pageview → intención de compra → checkout).
- **Sistemas**: Landing, PWA buy page, PostHog, ManyChat/IG.
 - **Aprendizaje vigente**: el muro de cuenta antes de pagar mata la conversión en webview de IG (blast 2026-07-05: ~100 visitas, 0 checkouts). El funnel de invitado es la respuesta.

1.2 Checkout y pago

1. Comprador firma con email → `POST /public/checkout/guest-start` (público, rate-limited) crea/encuentra el usuario en Firebase Auth del lado servidor.
 2. Ruteo de pasarela por país: **Colombia (CO)** → **MercadoPago**; **resto** → **Polar** (merchant-of-record, tarjetas internacionales USD). Override manual disponible.
 3. Precio internacional USD activo por defecto (derivado del precio COP).
 4. Webhook confirma el pago (MP: HMAC-SHA256; Polar: `handleOrderPaid`), con idempotencia vía `processed_payments`.
- **Sistemas**: Express API (`payments.ts`, `polar.ts`, `public.ts`), Firebase Auth, Firestore.
 - **Regla**: pagos siempre server-side, nunca desde cliente.

1.3 Onboarding / acceso

1. Post-pago se envía **magic link** por correo (Resend) — es la vía de entrada a la cuenta.
2. Se concede acceso: entrada en `users/{uid}.courses` (`status`, `access_duration`, `expires_at`, `deliveryType`, `title`).
3. Primera entrada: flujo de onboarding (`OnboardingFlow.web.jsx`, 13 pasos) captura perfil y objetivos.

4. Home (`HoyScreen`) muestra los programas del coach: carrusel workout → nutrición → semana/coach.

- **Aprendizaje vigente (identidad):** un humano = una cuenta. Riesgo: pagar con correo X y entrar con Google correo Y crea cuenta paralela (doble cobro / comprador varado). Mitigaciones vivas: correos + pantalla post-compra con "Abrir Wake" / "Gestionar suscripción" / WhatsApp, badge "Última vez" en login, guía según correo. Pendiente: "claim on entry" server-side.

1.4 Soporte y éxito del cliente

1. Canales: WhatsApp (`wa.me/573178751956`) y correo (`soporte@wakelab.co` → ImprovMX → Gmail).
 2. Casos típicos: no puedo entrar, me cobraron doble, no veo mi programa.
 3. Remediación manual con scripts de admin (reembolsos, reasignar acceso, unir cuentas).
- **Sistemas:** Gmail, WhatsApp, scripts en `scripts/`.

1.5 Renovación / cancelación

1. Suscripciones recurrentes: MP (PreApproval) y Polar (portal).
2. Cancelar: MP → en la app; Polar → un botón único "Gestionar suscripción" que abre el portal Polar.
3. Feedback de cancelación se guarda en `subscription_cancellation_feedback`.

2. Contenido — programas de atletas

2.1 Alta de un atleta

1. Se crea/eleva un usuario a `role: creator`.
2. Accede al **Creator Dashboard** (`/creators`) para gestionar clientes, programas, libraries, nutrición y bookings.

2.2 Creación de un programa

1. El atleta arma el árbol del programa: `courses` → `modules` → `sessions` → `exercises` → `sets`.
2. `deliveryType` define el modelo de entrega:
 - `general` — árbol fijo para todos (incluye el antiguo `low_ticket`, ya consolidado).
 - `one_on_one` — semana a semana, por cliente (`plans/...`), con tira de 7 días.
3. Reutiliza contenido desde sus **libraries** (`creator_libraries/{creatorId}/...`).
4. Nutrición: crea comidas y planes en `creator_nutrition_library`, los asigna a clientes (`nutrition_assignments`).
5. Recursos extra (PDF/YouTube/link) por programa: `additional_resources` → card "Recursos".

2.3 Publicación

1. El programa nace en **borrador**; se publica manualmente.

2. Al publicar se activa el precio internacional USD en Polar (derivado del COP).
3. Opcional: **cap de compra + waitlist** por programa (`courses.capacity`), y **secciones de landing** autoradactadas (`landing_sections`) para su buy page.

2.4 Modelo de drops mensuales (caso Bejarano)

1. Un programa general con contenido que se libera por **drops mensuales** anclados a calendario.
 2. Cron cohort-sync (primer lunes) libera el mes; acceso rodante de 30 días por usuario.
 3. Política de lapso: se conserva el historial, se bloquea el programa.
- **Sistemas:** Creator Dashboard, Firestore (`courses` , `plans` , `creator_libraries` , `creator_nutrition_library`), cron `monthlyDropAdvance` .
-

3. Producto / Ingeniería — construir y operar

3.1 Build & deploy

1. Monorepo: 3 apps web (`landing` , `pwa` , `creator-dashboard`) + Cloud Functions, un solo Firebase Hosting.
 2. Build: `npm run build:all` (o por app) → `npm run assemble-hosting` .
 3. Deploy: `firebase deploy` (`--only functions` / `--only hosting`). **Confirmar siempre antes de desplegar — wolf-20b8b es producción.**
 4. Alternativo en la nube: GitHub Actions → "Deploy a producción" (`deploy-prod.yml`).
- **Gotchas vivos:** `assemble-hosting` **borra** `hosting/` (para deploy solo-landing, overlay con `rsync -a apps/landing/dist/ hosting/`); un `firebase deploy` puede auto-commit + push a `main` ; el output de creators es `build/` no `dist/` .

3.2 Arquitectura de datos

- Todo dato pasa por **service singletons** y la **API Express** (`/api/v1/*`); componentes nunca tocan el SDK de Firestore directo.
- Data fetching solo con **React Query** (staleTime desde `queryConfig.js`); sin `onSnapshot` .
- Un solo archivo de exports de funciones: `functions/src/index.ts` .

3.3 Observabilidad — Wake Ops

1. Colectores tontos + agente inteligente sobre bus de Telegram (`docs/WAKE_OPS.md`).
2. Análítica de producto: PostHog (funnels, error tracking con reglas de supresión de ruido).

3.4 Manejo de incidentes

1. Detección: alertas de Wake Ops / reportes de usuario / errores en PostHog.
2. Diagnóstico: revisar logs (`functions.logger`), estado de deploy, memorias de incidentes previos.
3. Fix → verificar en prod → commit → memoria del incidente si es reutilizable.

- **Ejemplo vivo:** epidemia de OOM en funciones v2 256MiB (2026-07-05), resuelta subiendo a 512MiB.

3.5 Móvil

- Builds nativos vía EAS (bundle `com.lab.wake.co`); la PWA es el export web de Expo.
-

4. Finanzas — plata que entra y sale

4.1 Ingresos

1. Cada venta/renovación cae por webhook de MP o Polar y queda en el `collectionGroup` `subscriptions` (`transaction_amount`, `status`).
2. En cada venta/renovación el atleta dueño del programa recibe un correo "Nueva venta"/"Renovación" (`sendCreatorSaleNotification`). No en trials/refunds; no en autocompra.

4.2 Fees de pasarela (reales)

- **MercadoPago:** ~5.12%.
- **Polar:** 5% + US\$0.50 (plan Starter). Tarjetas únicamente → por eso CO se queda en MP.
- Impuesto: jurisdicción del comprador (no IVA CO).

4.3 Comisiones y payouts a atletas

1. Reparto Wake ↔ atleta sobre el monto **bruto** de cada venta.
2. `PLATFORM_COMMISSION_RATE` aún **null** (pendiente de fijar) — el split formal automático es trabajo futuro.
3. Modelo futuro de split-at-source en evaluación (dLocal vs. Stripe Atlas + Connect).

4.4 Refunds y ajustes

1. Refunds manuales por pasarela (MP mantiene `status=approved` en refund parcial → no revoca acceso; Polar usa `next_period` `pending_update`).
2. Cambios de precio a suscriptores activos: migrar cada suscripción + reembolsar diferencia + notificar (precedente: reprice Bejarano 79k→19k, 2026-07-06).

4.5 Costos de infraestructura

1. Facturación real en BigQuery (`billing_export`); factura mensual ~US\$1.75 equiv.
2. Costo dominado por conteos fijos: versiones de Secret Manager + jobs de Cloud Scheduler (Wake Ops). Firestore/compute/egress en free-tier.

4.6 Herramienta de foto financiera

- `scripts/finance-snapshot.js` (READ-ONLY): estado de drops + suscriptores + ventas MP/Polar + checkouts abandonados + auditoría de huérfanos Polar.

Proveedores clave

Proveedor	Para qué	Notas
Firebase (<code>wolf-20b8b</code>)	Auth, Firestore, Storage, Hosting, Functions	Producción
MercadoPago	Cobros Colombia (COP)	Server-side; se reemplazará a futuro
Polar	Cobros internacionales (USD), merchant-of-record	Solo tarjetas
Resend	Envío de correo transaccional	Desde <code>hola@wakelab.co</code>
ImprovMX	Recepción de correo <code>wakelab.co</code>	Forward → Gmail
FatSecret	Base de datos de nutrición	Solo vía proxy en Functions
PostHog	Analítica de producto y error tracking	—
GoDaddy	DNS de <code>wakelab.co</code>	API key en Secret Manager

Referencias

- Estructura de la empresa: `docs/ORGANIGRAMA.md`
- Principios de ingeniería y arquitectura: `CLAUDE.md`
- Sistema visual / UI: `docs/STANDARDS.md`
- Marca y voz: `docs/BRAND.md`
- Endpoints de la API: `docs/API_ENDPOINTS.md`
- Trabajo pendiente: `docs/PENDING_WORK.md`
- Observabilidad: `docs/WAKE_OPS.md`